DESARROLLO CON PYTHON

Contenido de formación


Programación


Pyhton


Paradigma POO


Bases de datos SQL


Tkinter

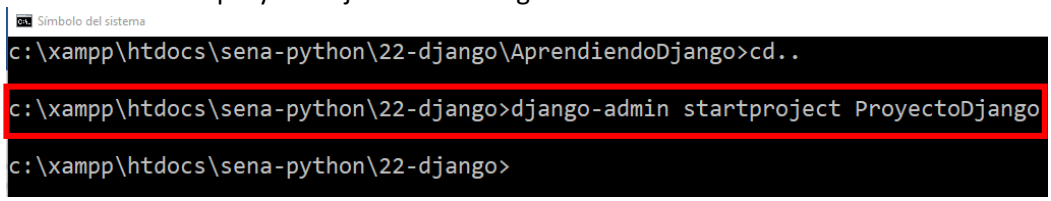

Django

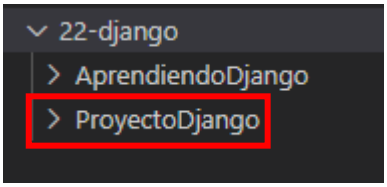

Flask

1. Primeros Instalación de SW
2. Variables y tipos de datos
3. Operadores (aritméticos, asignación)
4. Entrada y salida de datos
5. Estructuras de control
   5.1 Condicionales (operadores lógicos, operadores de comparación)
   5.2 Bucles (estructuras interactivas for, while)
6. Bloque de ejercicios de aplicación de conceptos
7. Funciones (parámetros, return, invocación, lambda)
   7.1 Variables locales y globales, funciones y métodos predefinidos
8. Lista y tuplas (creación, índices, recorrer y mostrar listas, listas multidimensionales)
9. Diccionarios y sets
10. Bloque de ejercicios aplicación de conceptos
11. Módulos y paquetes (creación y funcionalidad)
12. Sistemas de archivos y directorios
13. Manejo de errores (captura de excepciones, errores personalizados)
14. Programación orientada a objetos
15. Bases de datos SQLite
16. Bases de datos MySQL
17. Proyecto con Python
18. Interfaces graficas con Tkinter (aplicación de escritorio Python – Tkinter)
19. Desarrollo Web con Django
20. Interfaces graficas con Flask (aplicación de escritorio Python – Flask)

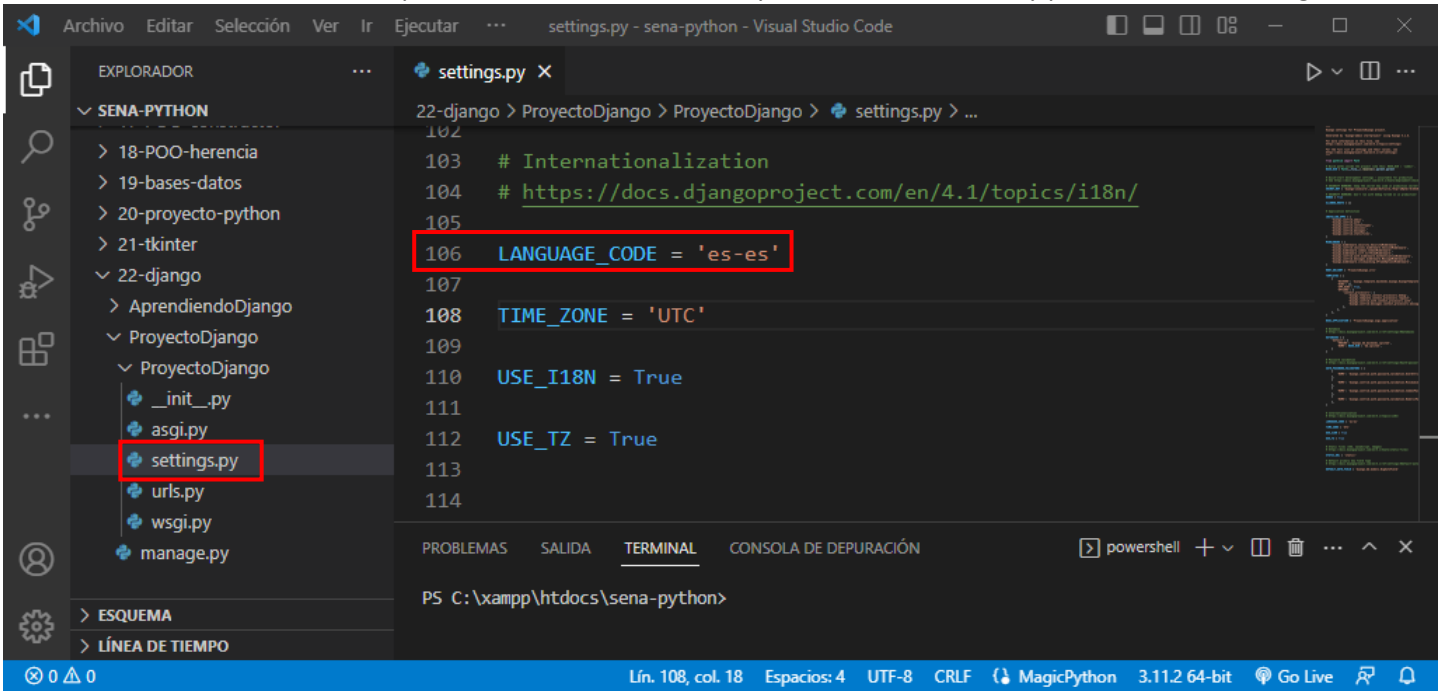
Proyecto Django Completo

Creamos nuestro proyecto ejecutando la siguiente ruta

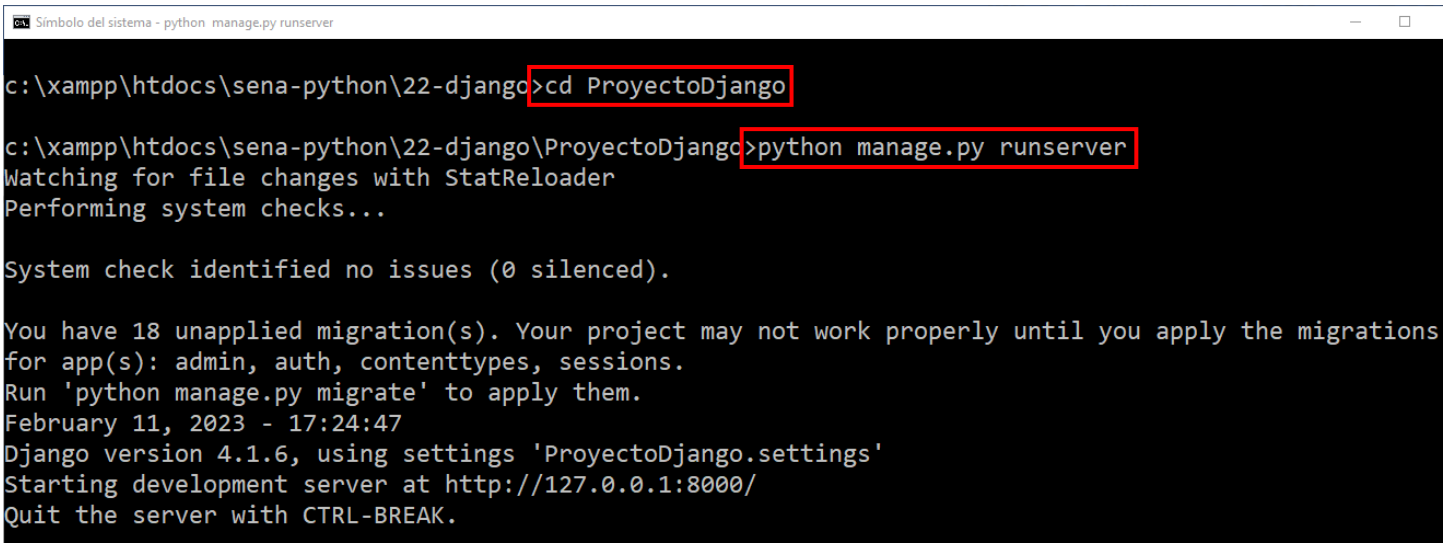




Poner el **framework** en modo español realizamos los cambios y corremos el servidor y probamos en el navegador



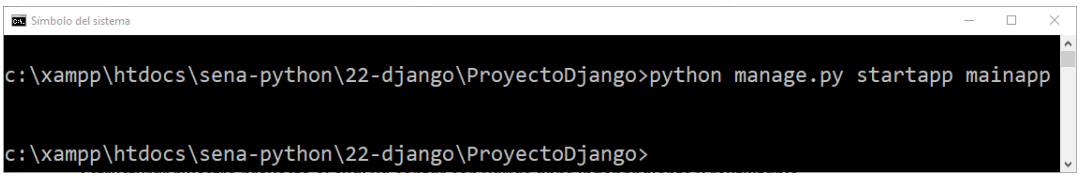
Corremos el servidor

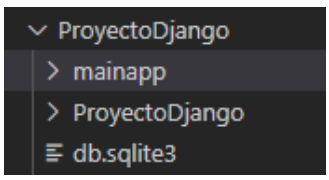




Estructurar Nuestro proyecto el cual va contar con varias apps independiente y reutilizable

Creamos nuestra app nueva la cual tendrá el nombre de **mainapp** y se verifica la creación en el proyecto





Crear vistas, urls y templetes

Preparamos la aplicación principal – en **settings.py** dentro de **ProyectoDjango** vamos al listado en **INSTALLED_APPS** añadir ***'mainapp'***

a. En settings.py adicionar la app

```
33  INSTALLED_APPS = [
34      'django.contrib.admin',
35      'django.contrib.auth',
36      'django.contrib.contenttypes',
37      'django.contrib.sessions',
38      'django.contrib.messages',
39      'django.contrib.staticfiles',
40      'mainapp'
```

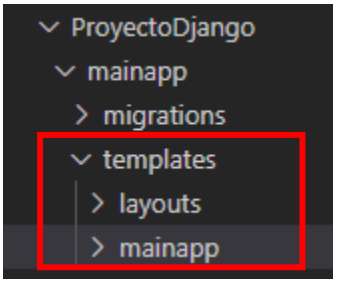
b. Crear definiendo una vista nueva en ***views.py*** - **def index** y se le pasa la **(request)** y devolvemos algo con **return render y** devolvemos una vista que estará guardada en una carpeta que se llamara templates

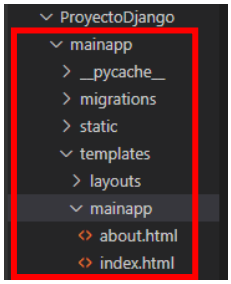
views.py

22-django > ProyectoDjango > mainapp > views.py > index

```
1  from django.shortcuts import render
2
3  # Create your views here.
4
5  def index(request):
6      return render(request, 'mainapp/index.html',{
7          'title': 'Inicio'
8      })
9
```

c. Creamos 1 carpeta de nombre **templates** dentro creamos una carpeta con el nombre **mainapp** y otra **layouts** dentro de **mainapp** creamos index.html





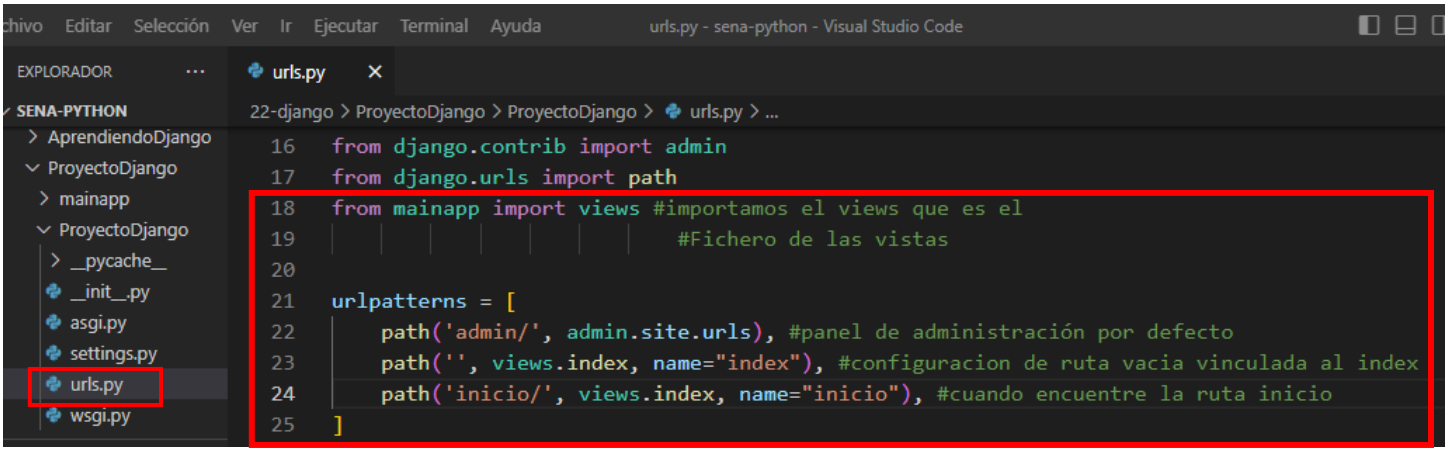
d. Ahora creamos el template dentro de la carpeta mainapp se creará el fichero **index.html** y pintamos la variable **title** que se ha pasado desde la vista.

index.html

22-django > ProyectoDjango > mainapp > templates > mainapp > index.html

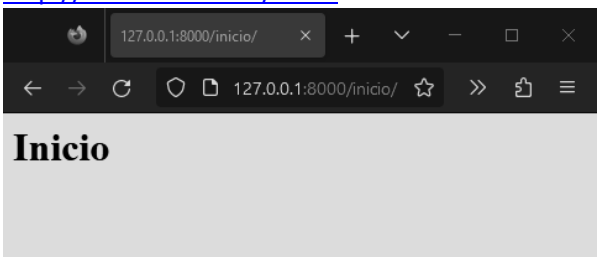
```
1  <h1>{{title}}</h1>
```

e. Para que funcione debo crear una nueva URL dentro de carpeta ***ProyectoDjango*** y voy al fichero ***urls.py*** Importamos **from mainapp import views**

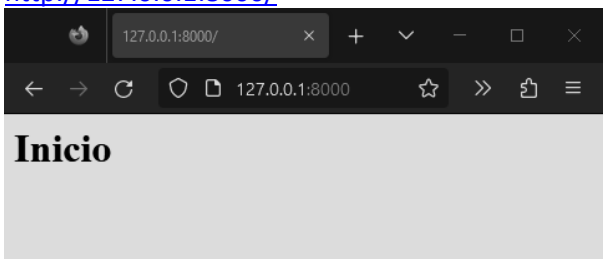


La prueba en los dos casos:

http://127.0.0.1:8000/inicio

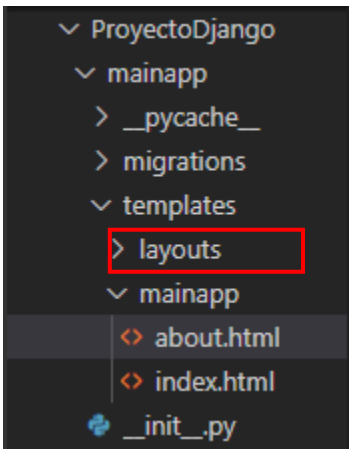


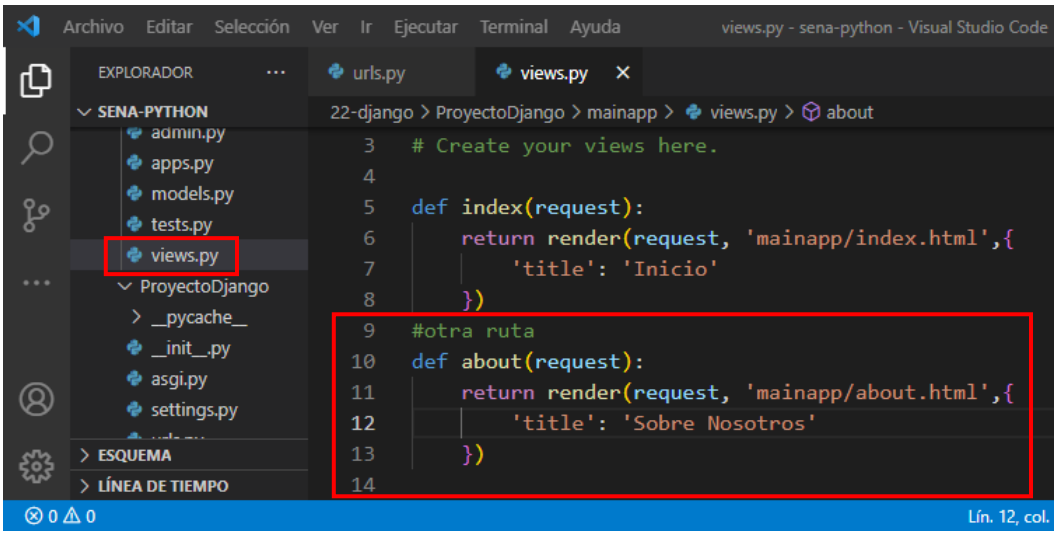
http://127.0.0.1:8000/

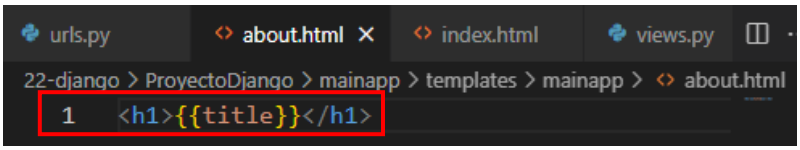


Creación de otra vista:

Creamos un fichero ***about.html*** en ***mainapp*** y luego la vista en ***views.py***



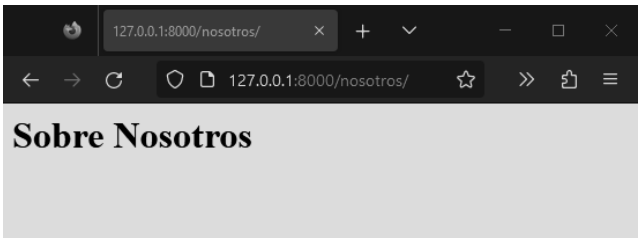




Ahora creamos la url – ***urls.py***

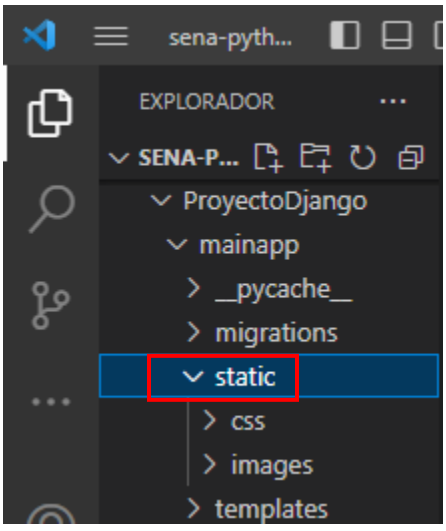
```python
from django.contrib import admin
from django.urls import path
from mainapp import views #importamos el views que es el
                          #Fichero de las vistas

urlpatterns = [
    path('admin/', admin.site.urls), #panel de administración por defecto
    path('', views.index, name="index"), #configuracion de ruta vacia vinculad
    path('inicio/', views.index, name="inicio"), #cuando encuentre la ruta ini
    path('nosotros/', views.about, name="about"), #Creamos la url para about
]
```



Creando estilos al proyecto: Maquetación

Para agilizar el proceso vamos a traer todo el diseño que teníamos en el proyecto anterior entonces copiamos la carpeta **static** y la pegamos dentro de **mainapp**



También copiamos el **layout** que se uso en el proyecto anterior y la pegamos en **templates→layouts→layout.html**
Y realizamos algunos cambios:



layout.html

22-django > ProyectoDjango > mainapp > templates > layouts > layout.html

```
{% load static %}
<!DOCTYPE html>
<html>
    <html lang="es">
    <head>
        <meta charset="utf-8">
        <title>
            {% block title  %}
            <!-- Titulo de cada pagina en concreto-->
            {% endblock  %}
            |- ADSO SENA
        </title>
        {% load static %}<!--carga de ficheros estaticos-->
        <link rel="stylesheet" type="text/css" href="{%static 'css/styles.css' %}"/>
    </head>
    <body>
        <!-- Cabecera de la Página -->
        <header>
            <div id="logotipo"> <!--se usa id porque sera algo que estara una vez en la pagina-->
                <!--Logo Django-->
                <img src="{% static 'images/logo.png' %}"/ >
                <h1> Proyecto con Django </h1>
            </div>
        </header>
        <nav>
            <ul>
                <li>
                    <a href="{% url 'index' %}">Inicio</a>
                </li>
                <li>
                    <a href="{% url 'about' %}">Sobre Nosotros</a>
                </li>
            </ul>
        </nav>
            <div id="content">
                <div class="box">
                    <h2 class= "title"></h2>
                    {%block content %}
                        <strong>Contenido del bloque original!!</strong>
                    {% endblock %}
                </div>
            </div>
        <footer>
            Programación con Django - ADSO 2023 &copy; Instructor Moisés Garcia - {% now "d/m/Y h:m:
        </footer>
    </body>
</html>

```

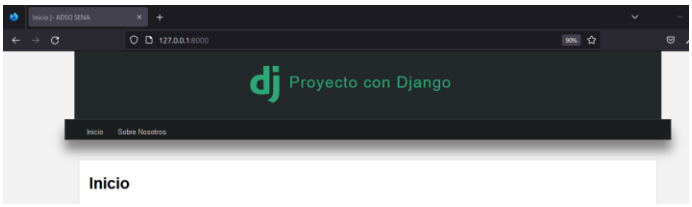
Ahora usamos el **layout** en **index.html** y en **about.html** para lo cual heredamos esa layout

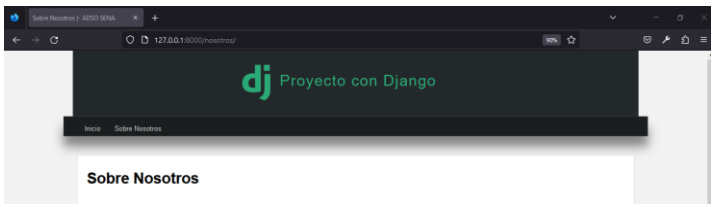
index.html

```
22-django > ProyectoDjango > mainapp > templates > mainapp > index.html
1  {% extends 'layouts/layout.html' %}
2  {% block title %} {{title}} {% endblock  %}
3
4  {% block content %}
5      <h1>{{title}}</h1>
6  {% endblock  %}
```

about.html

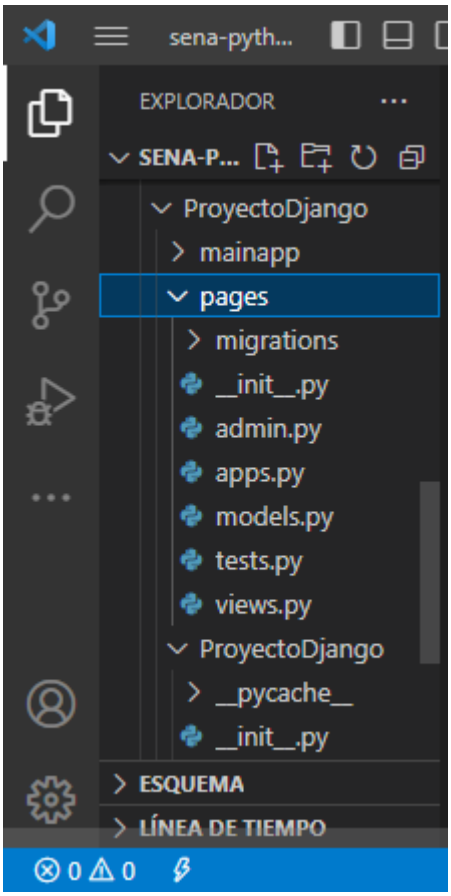
```
22-django > ProyectoDjango > mainapp > templates > mainapp > about.html
1  {% extends 'layouts/layout.html' %}
2  {% block title %} {{title}} {% endblock  %}
3
4  {% block content %}
5      <h1>{{title}}</h1>
6  {% endblock  %}
```





Creación de otra app para el proyecto

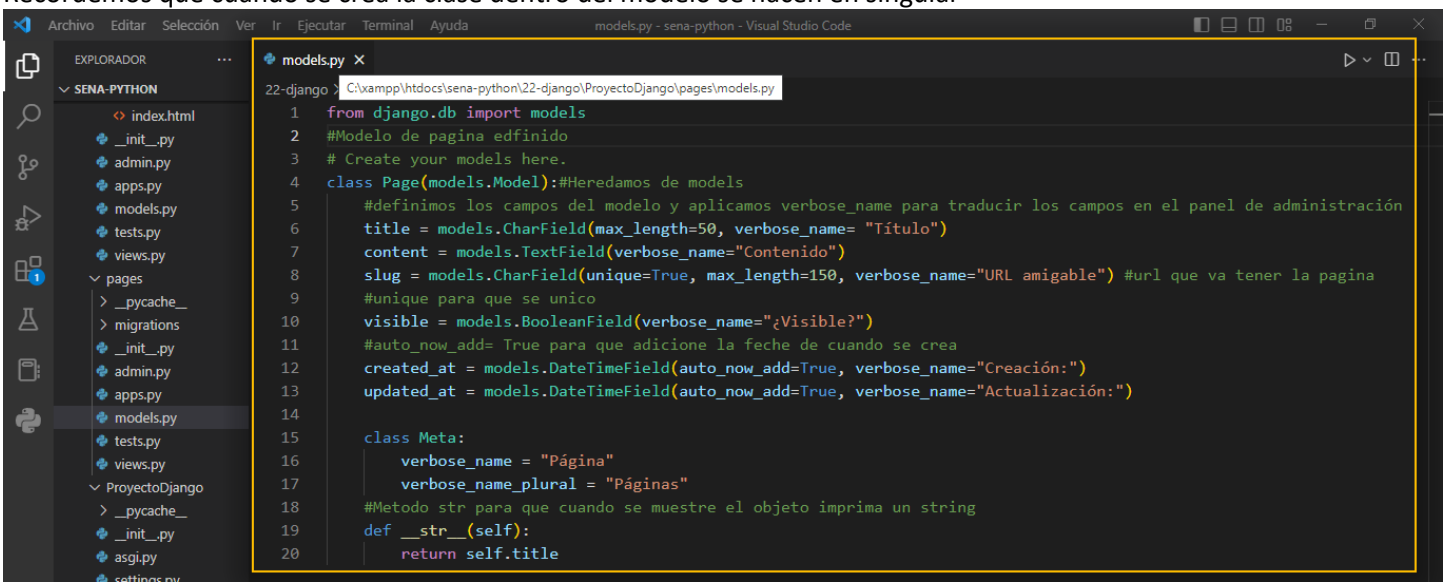
Se creará otra app para las páginas, lo que permitirá gestionar diferentes páginas dentro de nuestro proyecto, para lo cual detenemos el servidor y ejecutamos el comando para crear una app llamada **pages**. Si no recuerda cual es el comando, consultar el material anterior.



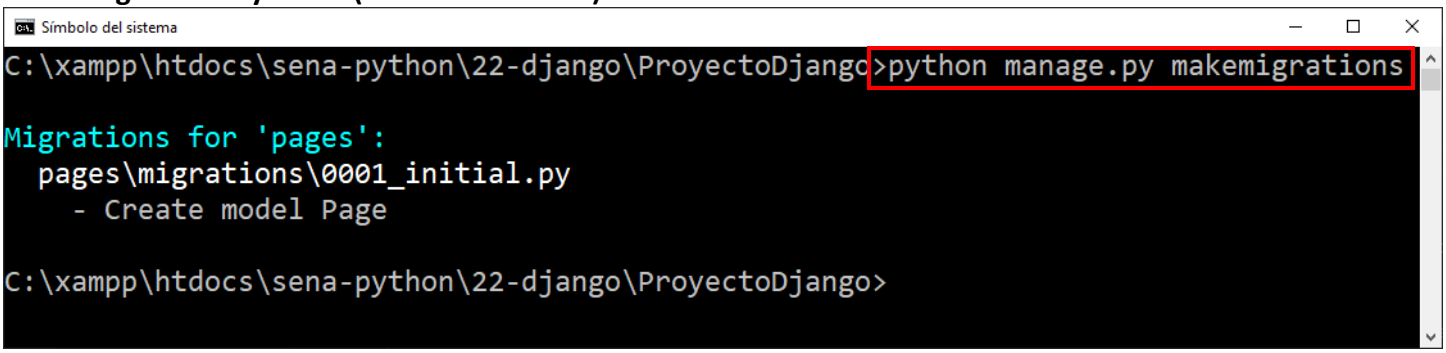
Ahora para trabajar con la aplicación pages debe añadirlo **INSTALLED_APPS**

Creación de modelos:

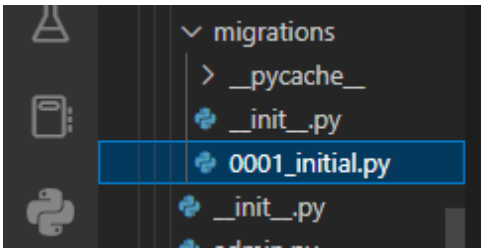
Recordemos que cuando se crea la clase dentro del modelo se hacen en singular



Crear migraciones y tablas(creación de la BD)



Cuando se realicen cambios en los modelos se debe crear una nueva migración



Ahora ejecutamos el comando **python manage.py sqlmigrate pages 0001** para crear el fichero sql que se ejecutara internamente cuando migre todo

Y genera el código sql que se ejecutara.



```
Símbolo del sistema
C:\xampp\htdocs\sena-python\22-django\ProyectoDjango>Python manage.py sqlmigrate pages 0001

BEGIN;
--
-- Create model Page
--
CREATE TABLE "pages_page" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "title" varchar
(50) NOT NULL, "content" text NOT NULL, "slug" varchar(150) NOT NULL UNIQUE, "visible" bool
 NOT NULL, "created_at" datetime NOT NULL, "updated_at" datetime NOT NULL);
COMMIT;

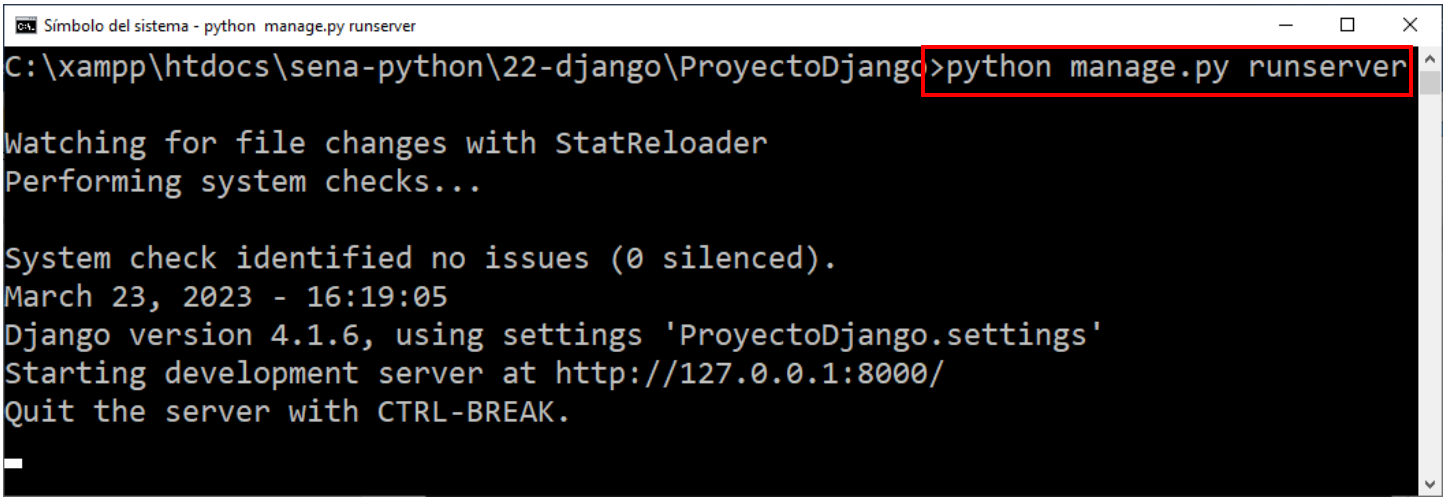
C:\xampp\htdocs\sena-python\22-django\ProyectoDjango>
```

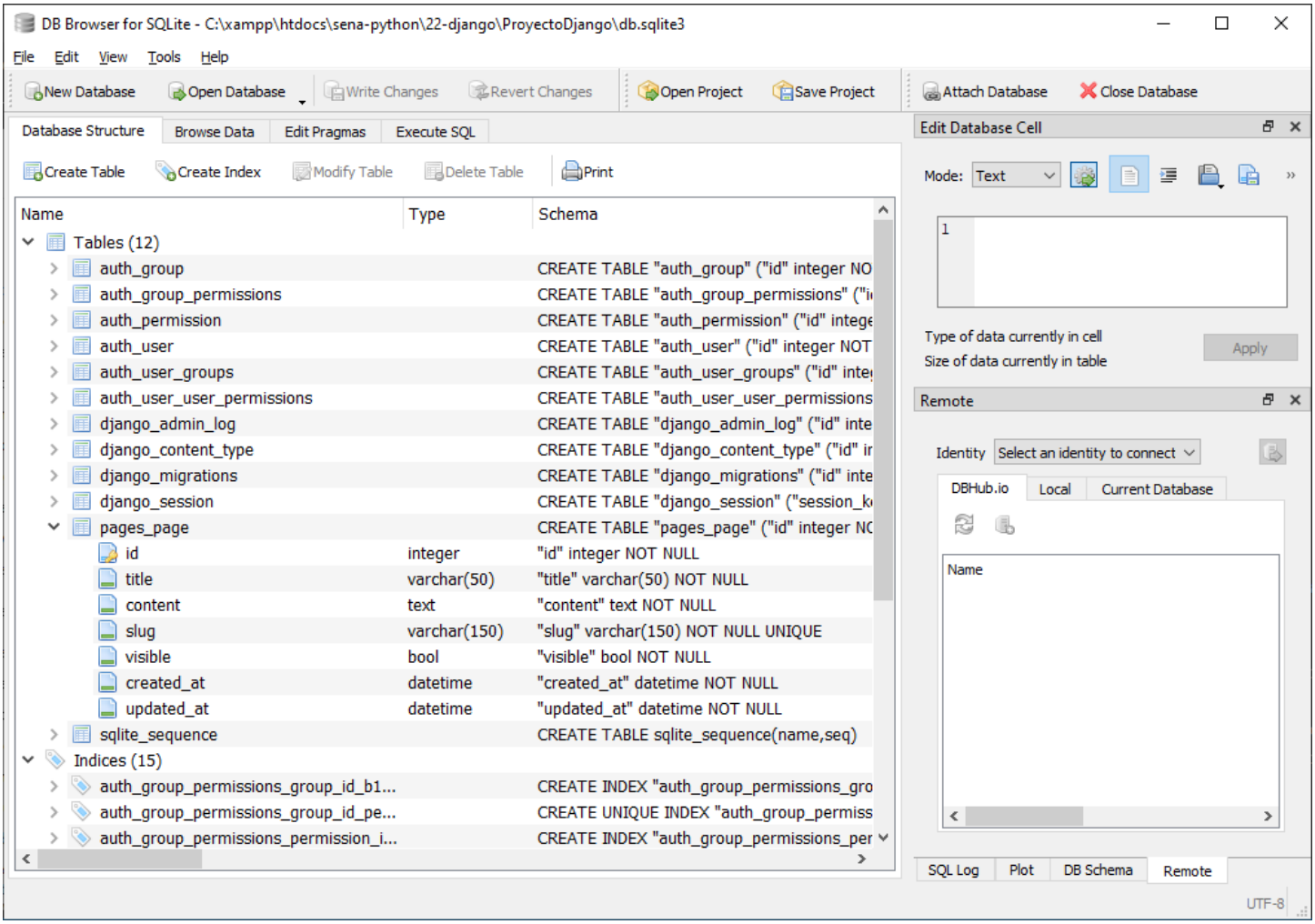
Ahora si ejecutamos la migración con el comando **python manage.py migrate**

```
Símbolo del sistema
C:\xampp\htdocs\sena-python\22-django\ProyectoDjango>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, pages, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying pages.0001_initial... OK
  Applying sessions.0001_initial... OK
```

En estos momentos ya tenemos la base de datos creada y procedemos a arrancar el servidor y comprobamos que ha generado la base de datos en db browers.


python
django
Flask
web development, one drop at a time
TKINTER
MySQL
SQLite


Símbolo del sistema - python manage.py runserver
C:\xampp\htdocs\sena-python\22-django\ProyectoDjango>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
March 23, 2023 - 16:19:05
Django version 4.1.6, using settings 'ProyectoDjango.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.


DB Browser for SQLite - C:\xampp\htdocs\sena-python\22-django\ProyectoDjango\db.sqlite3
File Edit View Tools Help
New Database
Open Database
Write Changes
Revert Changes
Open Project
Save Project
Attach Database
Close Database
Database Structure
Browse Data
Edit Pragmas
Execute SQL
Create Table
Create Index
Modify Table
Delete Table
Print
Name
Type
Schema
Tables (12)
auth_group
auth_group_permissions
auth_permission
auth_user
auth_user_groups
auth_user_user_permissions
django_admin_log
django_content_type
django_migrations
django_session
pages_page
id integer "id" integer NOT NULL
title varchar(50) "title" varchar(50) NOT NULL
content text "content" text NOT NULL
slug varchar(150) "slug" varchar(150) NOT NULL UNIQUE
visible bool "visible" bool NOT NULL
created_at datetime "created_at" datetime NOT NULL
updated_at datetime "updated_at" datetime NOT NULL
sqlite_sequence
Indices (15)
Edit Database Cell
Mode: Text
Type of data currently in cell
Size of data currently in table
Apply
Remote
Identity
Select an identity to connect
DBHub.io
Local
Current Database
Name
SQL Log
Plot
DB Schema
Remote
UTF-8